

PONTIFICIA UNIVERSIDAD JAVERIANA
Facultad de Ingeniería — Introducción a Sistemas Distribuidos

Sistema Reservas de Boletos

Primera entrega — Arquitectura base

Integrantes:
Elier David Ibarra Medina
Samuel Adrian Montaña Linares

Profesor: Rafael Vicente Paez Mendez

1 Introducción

El Sistema de Reserva de Boletos es una aplicación distribuida que permite a los estudiantes de la universidad consultar los eventos culturales y deportivos de un mes, reservar puestos y, más adelante, modificar o cancelar sus reservas. Se organiza en tres capas (consumidores, lógica de negocio y almacenamiento), cada una en una máquina distinta, que se comunican con ZeroMQ combinando patrones síncronos (consulta y reserva) y asíncronos (cancelación).

Esta primera entrega cubre la arquitectura base: la consulta y la reserva funcionan de extremo a extremo (cliente → Gestor de Transacciones → RMC → Persistencia), sin tolerancia a fallas. El informe presenta los cuatro modelos del sistema, cada uno con los diagramas de diseño que le corresponden, y define el protocolo de pruebas, la forma de obtener las métricas y el mecanismo de generación de carga. La cancelación, la modificación, la escalabilidad y la tolerancia a fallas se diseñan aquí pero se implementan en la segunda entrega.

2 Modelos del Sistema

2.1 Modelo Físico

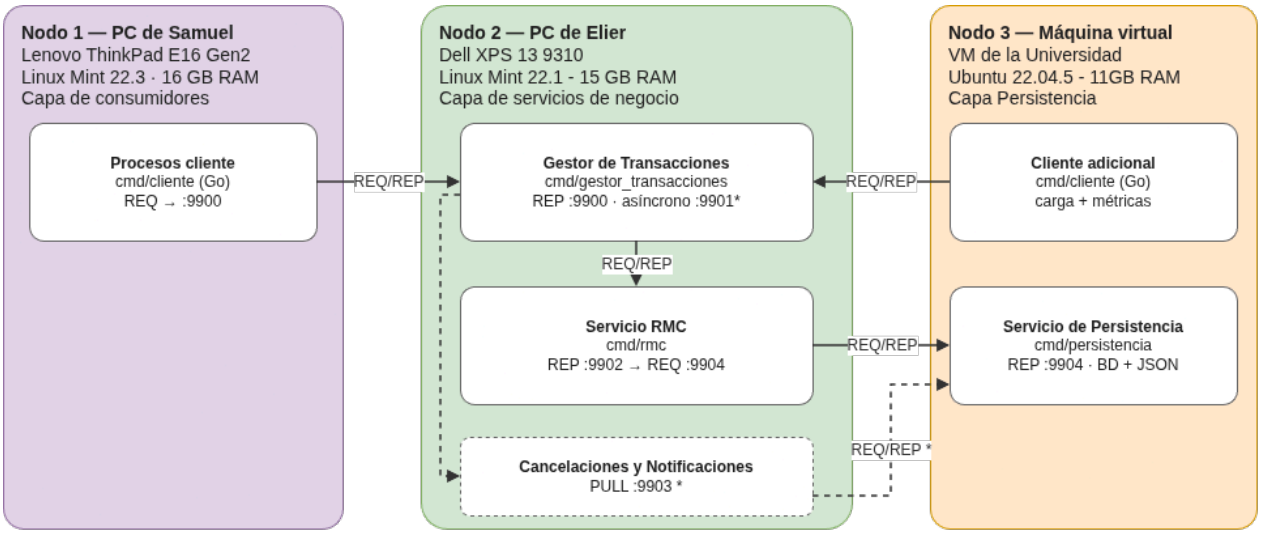


Figura 1: Diagrama de despliegue: nodos, componentes, puertos y comunicación entre capas.

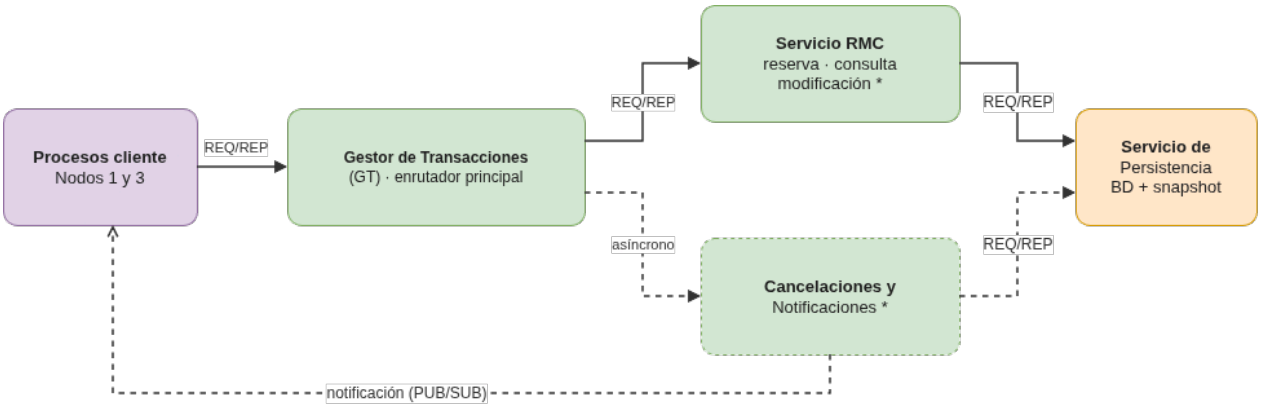


Figura 2: Diagrama de componentes: servicios del sistema y sus interfaces de comunicación.

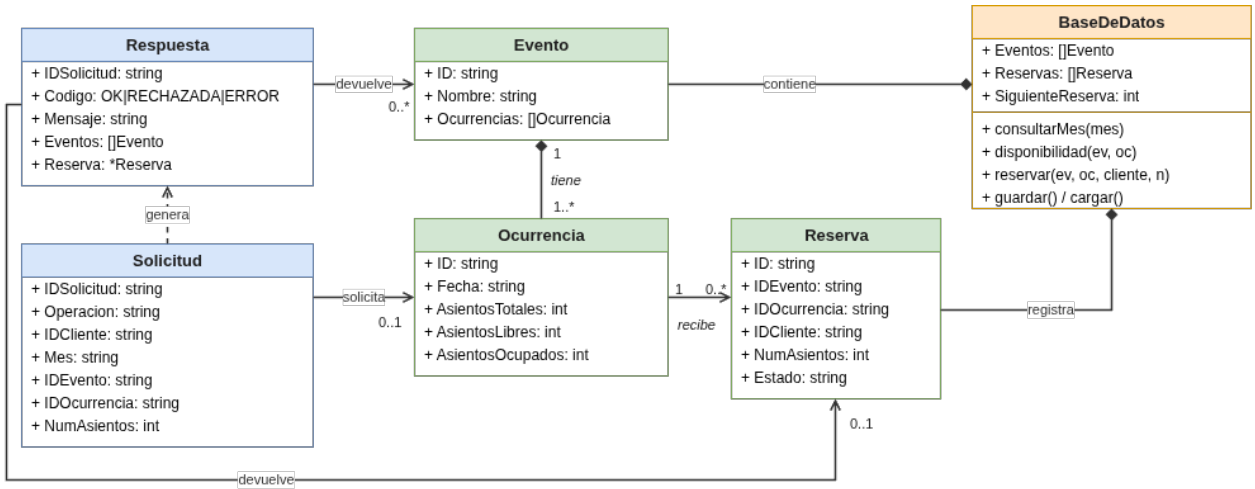


Figura 3: Diagrama de clases: modelo de datos, mensajes del protocolo y almacén de la persistencia.

Modelo físico. El sistema se despliega en tres dispositivos (Figura 1), uno por capa. Como no están en la misma red física, se conectan mediante **Tailscale**, una red privada virtual que les asigna direcciones IP fijas sin importar desde dónde se conecten.

Nodo	Dispositivo	Sistema operativo	RAM / disco	IP	Rol
1	Lenovo ThinkPad E16 Gen2 (Samuel Montaña)	Linux Mint 22.3	16 GB / 512 GB	100.91.50.27	Capa de consumidores: procesos cliente que generan las solicitudes de consulta y reserva.
2	Dell XPS 13 9310 (Elier Ibarra)	Linux Mint 22.1	15 GB / 229 GB	100.100.235.31	Capa de negocio: Gestor de Transacciones, RMC y (Entrega 2) Cancelaciones y Notificaciones.
3	Máquina Virtual (VM)	Ubuntu 22.04.5	11 GB / 68 GB	100.93.109.37	Capa de almacenamiento: Servicio de Persistencia, más un cliente adicional que genera carga y mide sus tiempos de respuesta.

Modelo arquitectónico. Es una arquitectura cliente-servidor de tres capas (Figura 2). Los **procesos cliente** envían solicitudes al **Gestor de Transacciones** (GT), la puerta de entrada, que las enruta por el canal síncrono hacia el **RMC** (reserva, modificación y consulta); el RMC consulta y actualiza los datos a través del **Servicio de Persistencia**. El servicio de Cancelaciones y su canal asíncrono forman parte del diseño pero se implementan en la Entrega 2. Cada servicio tiene un puerto fijo para que la comunicación sea predecible:

Componente	Nodo	Puerto	Socket ZeroMQ
Gestor de Transacciones, canal síncrono	2	9900	REP (clientes se conectan con REQ)
Gestor de Transacciones, canal asíncrono	2	9901	ROUTER (Entrega 2)
Servicio RMC	2	9902	REP (el GT se conecta con REQ)
Cancelaciones y Notificaciones	2	9903	PULL / PUB (Entrega 2)
Servicio de Persistencia	3	9904	REP (el RMC se conecta con REQ)
Procesos cliente	1 y 3	—	REQ: solo hacen `connect`, no exponen puerto

El sistema está escrito en **Go** (versión 1.27.1) con la librería `go-zeromq/zmq4` (versión 0.17.0), escogidos por tres razones: las *goroutines* permiten simular muchos clientes concurrentes con poco costo; al ser compilado, sus tiempos de respuesta son consistentes para las métricas del proyecto; y la librería es Go puro, sin dependencias de C, de modo que cada servicio es un binario único que se copia a cada nodo. Los mensajes se serializan en JSON.

Datos. El modelo de datos (Figura 3) sigue el enunciado: 50 eventos en un mismo mes, 20 con dos ocurrencias y 30 con una, y 100 asientos por ocurrencia; los asientos se controlan por ocurrencia (libres + ocupados = totales). La persistencia mantiene los datos en memoria protegidos con un *mutex* y los guarda en un archivo JSON (*snapshot*), por lo que sobreviven a un reinicio. Cada respuesta lleva un código: `OK`, `RECHAZADA` (regla de negocio, como falta de cupo) o `ERROR` (falla del sistema).

2.2 Modelo de interacción

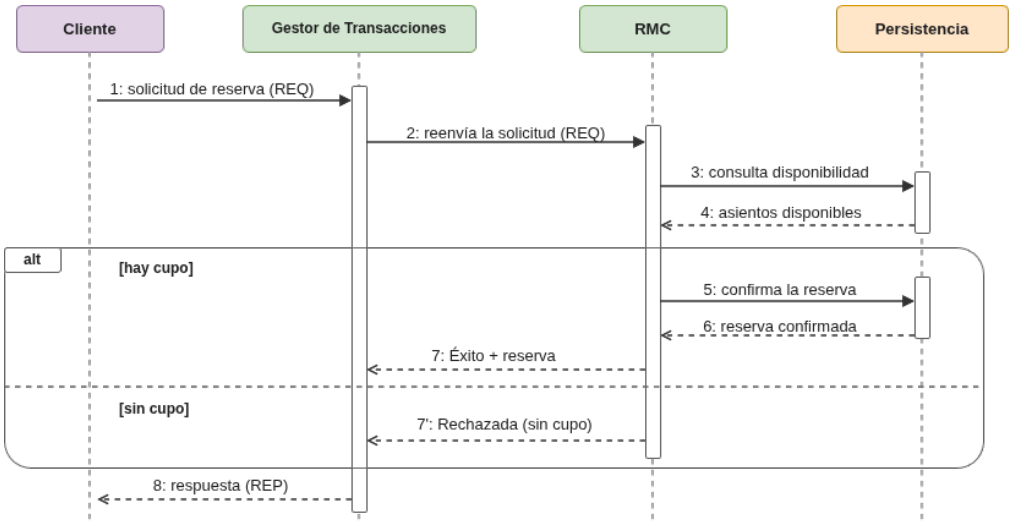


Figura 4: Secuencia de una reserva (síncrona, implementada): REQ/REP en los puertos 9900, 9902 y 9904.

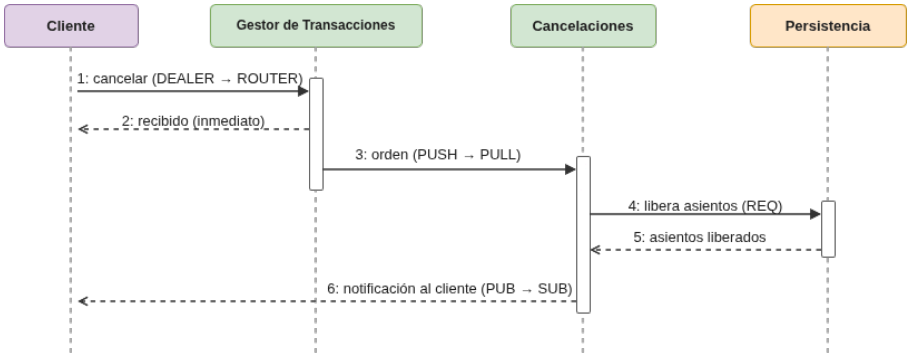


Figura 5: Secuencia de una cancelación (asíncrona, diseño): DEALER/ROUTER :9901, PUSH/PULL :9903, REQ :9904 y PUB/SUB.

Las interacciones se dividen según cómo espera la respuesta el cliente:

Interacción	Tipo	Patrón ZeroMQ
Cliente ⟶ GT (consulta, reserva)	Síncrona	REQ/REP
GT ⟶ RMC	Síncrona	REQ/REP
RMC ⟶ Persistencia	Síncrona	REQ/REP
Cliente ⟶ GT (cancelación) y confirmación inmediata	Asíncrona (Entrega 2)	DEALER/ROUTER
GT ⟶ Cancelaciones	Asíncrona (Entrega 2)	PUSH/PULL
Cancelaciones ⟶ Cliente (notificación)	Asíncrona (Entrega 2)	PUB/SUB

Operaciones síncronas. El cliente envía la solicitud y espera la respuesta antes de continuar. En una reserva (Figura 4) el RMC primero consulta a la persistencia la disponibilidad en tiempo real y, si hay cupo, confirma la reserva; la persistencia vuelve a validar el cupo y descuenta los asientos dentro de una sola sección crítica, por lo que dos clientes que compiten por los últimos puestos nunca producen sobreventa. La consulta sigue el mismo camino con un solo paso hacia la persistencia. Cada solicitud lleva un identificador único (`id_solicitud`) para trazarla en los registros de todos los servicios.

Operaciones asíncronas. En la cancelación (Figura 5) el GT confirma la recepción de inmediato y el cliente queda libre; cuando la persistencia libera los asientos, el servicio de Cancelaciones publica una notificación que el cliente recibe por separado.

El sistema es asíncrono en el sentido de los sistemas distribuidos: no hay cotas garantizadas de latencia entre nodos, porque el tráfico viaja por Tailscale. REQ/REP impone alternancia estricta (una solicitud a la vez por socket), por lo que el GT, el RMC y la persistencia atienden una solicitud a la vez; esta limitación es deliberada en la arquitectura base y es el punto que la arquitectura escalable debe resolver.

2.3 Modelo de fallos

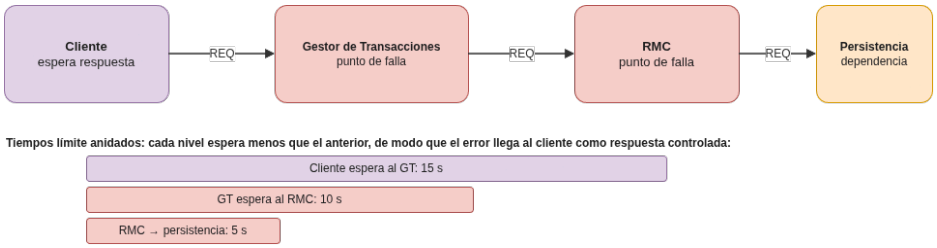


Figura 6: Diagrama del modelo de fallos: puntos de falla y tiempos límite anidados.

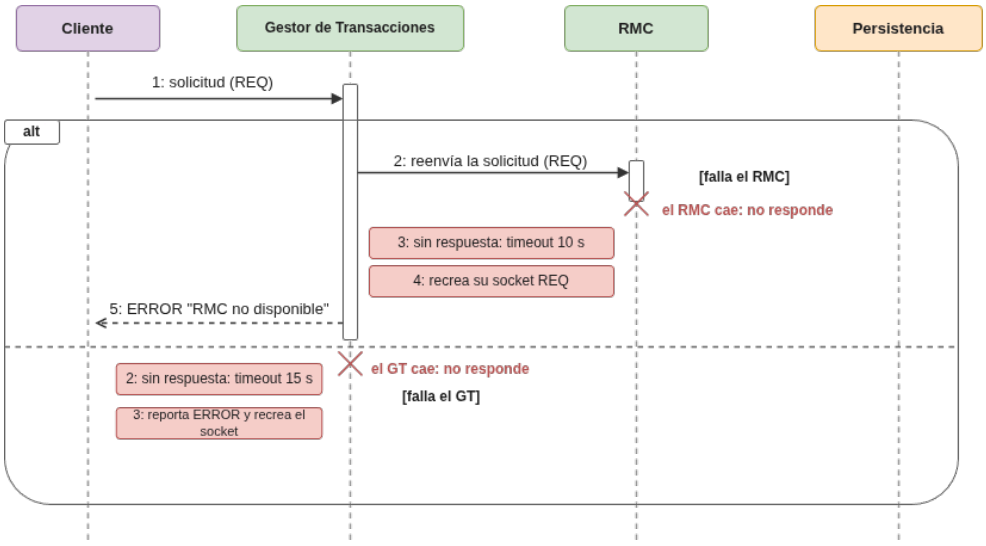


Figura 7: Secuencia de una solicitud cuando cae el RMC o el GT: instante de la caída y detección por timeout.

Se adopta el modelo de fallas por caída (*crash-stop*): un componente que falla deja de responder, pero no envía respuestas incorrectas ni corruptas. Los puntos de falla considerados son el Gestor de Transacciones y el RMC (Figura 6), responsables del flujo de consulta y reserva; la persistencia se trata como dependencia. También se contemplan fallas de omisión, es decir, mensajes perdidos por una interrupción de la conexión Tailscale, que desde fuera no se distinguen de una caída. No se consideran fallas arbitrarias (bizantinas).

La detección se basa en tiempos límite (*timeouts*) anidados: el cliente espera hasta 15 s al GT, el GT espera hasta 10 s al RMC y el RMC espera hasta 5 s a la persistencia. Como cada nivel espera menos que el anterior, una falla aguas abajo llega al cliente como una respuesta `ERROR` controlada y no como un bloqueo. La Figura 7 muestra el instante de la caída en los dos casos: si cae el RMC, el GT lo detecta a los 10 s, recrea su socket y responde `ERROR` al cliente; si cae el GT, es el cliente quien detecta el silencio a los 15 s. Tras un *timeout* el solicitante cierra su socket REQ y crea uno nuevo, pues un socket REQ queda bloqueado esperando una respuesta que ya no llegará. Es un detector imperfecto (no distingue una caída de una demora), con el compromiso habitual entre detectar rápido y evitar falsos positivos. En esta entrega solo se detecta y registra la falla; la recuperación se aborda en la Entrega 2.

2.4 Modelo de seguridad

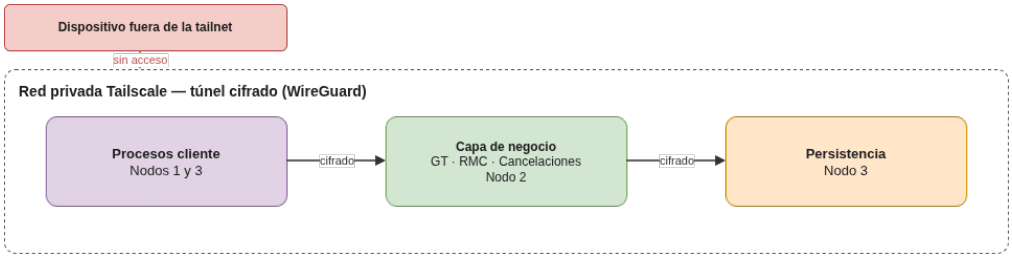


Figura 8: Diagrama del modelo de seguridad: perímetro de confianza y riesgos aceptados.

El perímetro de confianza es la red Tailscale (Figura 8): los tres nodos se comunican dentro de un túnel cifrado con WireGuard, y un dispositivo que no pertenece a la red no puede alcanzar los servicios. ZeroMQ no cifra por sí mismo, así que la **confidencialidad** y la **integridad** en tránsito dependen del túnel. No hay **autenticación** a nivel de aplicación: cualquier proceso dentro de la red Tailscale del equipo podría enviar solicitudes al GT. Como control

básico, cada servicio valida el formato de los mensajes y las reglas de negocio (mes, evento, ocurrencia y cantidad) y responde con un error controlado ante datos inválidos. Estos riesgos se aceptan porque el entorno de pruebas es controlado y solo el equipo tiene acceso a la red; como mejora se propone el mecanismo CURVE de ZeroMQ para autenticar clientes y cifrar a nivel de aplicación.

3 Arquitectura escalable y tolerante a fallas

Para la segunda entrega se propone: (i) *escalabilidad*: reemplazar REQ/REP por ROUTER/DEALER en el GT y colocar varias réplicas del RMC detrás de un balanceador, de modo que se atiendan solicitudes concurrentes, y replicar la base de datos con un esquema primario–réplica, enviando las consultas a las réplicas y las escrituras al primario; (ii) *tolerancia a fallas*: réplicas pasivas del GT y del RMC vigiladas con latidos (*heartbeats*), de modo que cuando se detecte una caída la réplica asuma el puerto del proceso caído. Para que las operaciones continúen en el punto en que se dejaron, el `id_solicitud` permite reintentos idempotentes y el estado de las operaciones en curso se registrará en la capa de persistencia; (iii) completar el canal asíncrono de cancelación (DEALER/ROUTER, PUSH/PULL y PUB/SUB).

4 Protocolo de pruebas

Tipo	Qué valida	Cómo	Criterio de éxito
Unitarias	Base de datos de la persistencia (carga inicial, consulta por mes, reserva, rechazos, guardado) y cálculo de métricas.	`go test ./cmd/... ./internal/...`	Pasan todas; en toda ocurrencia libres + ocupados = 100.
Integración	Flujo cliente → GT → RMC → Persistencia, y recuperación de la BD tras reiniciar la persistencia.	`go test ./tests/` compila y lanza los programas como procesos separados en localhost.	La consulta devuelve 50 eventos; la reserva descuenta asientos y queda guardada en el archivo JSON.
Funcionales	Consulta con y sin eventos, mes inválido, reserva válida, sin cupo, evento u ocurrencia inexistente, cantidad inválida, operación no soportada.	Cliente interactivo (`-interactivo`) sobre los tres nodos.	Cada caso devuelve `OK`, `RECHAZADA` o `ERROR` esperado y la BD queda como se espera.
Concurrencia	Varios clientes reservando la misma ocurrencia.	Prueba automática con 8 clientes y ejecución con 8 clientes en los nodos.	Nunca se venden más de 100 asientos.
Despliegue y red	Conectividad entre los tres nodos y arranque en el orden indicado.	`tailscale ping` y ejecución real en los tres dispositivos.	Todos los puertos son alcanzables y el flujo completo funciona.
Rendimiento	Escenarios de 3, 5 y 8 clientes (Tabla 1).	Generador de carga, 2–3 min por escenario, 3 repeticiones y BD reiniciada.	≥ 20 solicitudes/min por cliente y resultados repetibles.
Robustez	Detección de fallas sin bloqueo.	Detener el RMC o la persistencia durante la carga.	El cliente recibe `ERROR` en ≤ 15 s y el sistema sigue atendiendo.

5 Obtención de métricas de desempeño

Las métricas se obtienen instrumentando el código, sin herramientas externas de monitoreo, usando solo la biblioteca estándar de Go (`time`, `sync/atomic`, `math`, `encoding/csv`).

Métrica (Tabla 1)	Dónde se mide	Cómo
Tiempo promedio de respuesta y desviación estándar, de consultas y de reservas	Cliente (extremo a extremo)	Se toma `time.Now()` antes de enviar y `time.Since()` al recibir cada respuesta; cada solicitud se guarda en un CSV. La media y la desviación estándar muestral (dividiendo entre n-1) se calculan con `internal/metricas`. Las solicitudes con `ERROR` se cuentan aparte.
Transacciones de consulta y de reserva procesadas por minuto (TPM)	Gestor de Transacciones	Contadores por operación; TPM = transacciones procesadas / minutos entre la primera y la última. Se imprimen cada 15 s y al cerrar, y se guardan en `metricas_gt.csv`.
Estado de la base de datos	Persistencia	Resumen (eventos, asientos libres y ocupados, reservas) al iniciar, periódicamente y al cerrar, más el *snapshot* JSON.

Los CSV de los clientes de los nodos 1 y 3 se consolidan con `cmd/metricas`, un programa en Go que entrega la tabla final por escenario. Como apoyo se observa el uso de CPU y memoria con `htop`.

6 Generación de carga

La carga la produce un generador propio escrito en Go (`cmd/cliente`). No se usaron Locust ni JMeter porque están orientados a HTTP y los servicios hablan ZeroMQ; reutilizar el mismo protocolo y la misma librería evita traducciones que alterarían las mediciones. Cada cliente simulado es una *goroutine* con su propio socket REQ que envía solicitudes a un ritmo fijo, con un desfase inicial aleatorio. Antes de reservar, el cliente consulta el catálogo

para elegir ocurrencias existentes. Son parametrizables el número de clientes (`-clientes`), cuántos solo consultan (`-solo-consulta`), las solicitudes por minuto por cliente (`-rpm`, mínimo 20), el porcentaje de consultas de los clientes mixtos (`-pct-consulta`), la duración (`-duracion`), el mes y el máximo de asientos por reserva.

Escenario	**Perfil**	**Parámetros**
3 clientes	2 solo consulta + 1 consulta y reserva	`-clientes 3 -solo-consulta 2 -pct-consulta 50`
5 clientes	3 solo consulta + 2 reserva	`-clientes 5 -solo-consulta 3 -pct-consulta 0`
8 clientes	4 solo consulta + 4 reserva	`-clientes 8 -solo-consulta 4 -pct-consulta 0`

Los clientes se reparten entre el nodo 1 y el nodo 3, que ejecutan sus comandos al mismo tiempo. Como la modificación se implementa en la Entrega 2, los perfiles reserva y modificación usan solo reservas en esta entrega.